Database Systems - Study Notes

1. Why normalization matters

A relation schema is a set of tables and the rules that keep them consistent. Before a design can be normalized it has to be a valid relation: one where every cell holds a single value, every row is identified by a key, and every attribute depends on the key.

Normalization is the process of restructuring a database so that facts are stored once. Duplicate storage is not only wasteful, it creates update anomalies. If a student's name is stored on three separate rows, changing that name can leave the database in a state where the same person has two different names. Deleting the last row of a certain kind can delete facts about other kinds. Inserting a new row can be rejected because a fact that should not have been in that table is missing.

The goal is a design where each fact lives in exactly one place, and every non-key attribute depends only on the key of its own table.

2. Functional dependencies

A functional dependency is written X -> Y. It means that for any two rows in the relation, if the rows agree on all attributes in X then they must also agree on all attributes in Y. X is called a determinant and Y is called a dependent.

Some dependencies are trivial, meaning Y is a subset of X. For example in (StudentID, StudentName), the dependency StudentID -> StudentName is not trivial because StudentName is not part of StudentID.

Given a relation R with attributes A, B and C, the dependencies A -> B and A -> C together imply A -> BC. This is called transitivity of functional dependency. Dependency A -> B implies AB -> B, which is augmentation. Reflexivity says that any set of attributes determines itself.

A candidate key is a minimal set of attributes that determines all the other attributes in the relation. A primary key is the candidate key chosen to identify rows.

3. First, second and third normal form

First normal form, written 1NF, requires that every attribute holds a single atomic value and that every row is unique. A table that stores a comma separated list of phone numbers is not in 1NF. Nested tables break 1NF as well.

Second normal form, 2NF, requires 1NF and no partial dependency. A partial dependency exists when a non-key attribute depends on only part of a composite key. In a relation with the composite key (StudentID, CourseID) and the attributes StudentName and Grade, StudentName depends on StudentID alone. That is a partial dependency, so the relation is not in 2NF.

To fix 2NF you decompose the relation. Students move into their own table keyed by StudentID, and Enrolments keeps (StudentID, CourseID, Grade). StudentName now depends on the whole key of Students.

Third normal form, 3NF, requires 2NF and no transitive dependency of a non-key attribute on another non-key attribute. In a relation with key (StudentID, CourseID) and attributes CourseID, CourseName and InstructorID, the attribute CourseName depends on CourseID, and CourseID depends on the key. CourseName is therefore transitively dependent on the key, which violates 3NF.

The short version that is worth memorising for an exam: 1NF removes repeating groups, 2NF removes partial dependencies, and 3NF removes transitive dependencies.

4. BCNF and lossless joins

Boyce-Codd normal form, BCNF, is stricter than 3NF. In BCNF every nontrivial functional dependency X -> Y has X as a superkey. The difference between BCNF and 3NF shows up in relations with overlapping candidate keys.

Decomposition should be lossless, which means that joining the decomposed relations must return exactly the original rows, never fewer and never more. For a binary decomposition into R1 and R2 the decomposition is lossless if and only if the intersection R1 and R2 includes a key of at least one of the relations.

Decomposition should also be dependency preserving. The dependencies of the original relation should be checkable on the decomposed relations without joining them back together.

5. Indexes

An index is a data structure that lets the database find rows without scanning the whole table. A B-tree index keeps keys sorted and is useful for equality and range queries on frequently filtered columns.

Indexes speed up reads but every index must be updated on every insert, delete and update, so writes become slower. Indexes also consume disk space. A good strategy is to index columns used in WHERE clauses, join conditions and ORDER BY clauses.

A clustered index determines the physical order of the rows in the table, so a table can have at most one. A non-clustered index stores pointers back to the rows instead of moving them.

6. Referential integrity

A foreign key references a primary key or candidate key in another table, or sometimes in the same table. The rule is that the foreign key value must match an existing primary key value, or be null.

Inserts are checked against the parent table, and deletes or updates of the parent key are checked against the child table. Cascading actions automate this: ON DELETE CASCADE removes matching child rows, and ON DELETE SET NULL sets the foreign key to null instead.